

Adaptive Synthetic Data Augmentation Toolkit

Introduction: Modern ML often suffers when training data are scarce or imbalanced. Synthetic data generation can alleviate this, but static methods do not guarantee improved model accuracy. We propose a closed-loop toolkit that **dynamically generates synthetic tabular data** (or images) targeted at a model's current weaknesses. By analyzing model errors (e.g. on rare classes or outliers), the system synthesizes new data (via GANs, VAEs, copula models) to specifically address those areas, retraining the model in a feedback loop. This **adaptive augmentation** approach aims to boost performance and robustness beyond conventional static augmentation.

Problem Statement: Many ML applications (finance, healthcare, IoT) have limited or imbalanced tabular datasets. Traditional augmentation (adding noise or simple perturbations) is often ad-hoc and static. We address the unmet need for an **automated pipeline** that *intelligently decides what data to generate*, ensuring model improvement. The goal is to support a wide range of tasks by generating high-quality synthetic data tailored to the learning task.

Methodology:

1. **Data Analysis & Baseline Model:** Ingest a real dataset (e.g. healthcare records with imbalanced outcomes). Preprocess and train a baseline classifier/regressor (using scikit-learn or PyTorch). Evaluate baseline accuracy, fairness, and identify underperforming classes/cohorts via confusion analysis.
2. **Error Analysis:** Use tools or custom scripts to find cohorts with high error rates (e.g. minority classes, rare feature combinations).
3. **Data Generator Selection:** Integrate synthetic data engines (e.g. SDV for tabular, or a GAN/VAE for image). Based on error analysis, **parameterize the generator** to oversample the identified cohorts. For tabular data, methods may include conditional GANs or copulas; for images, a GAN conditioned on the problematic class.
4. **Closed-loop Generation:** Iteratively sample synthetic data, augment the training set, and retrain the model. After each round, re-evaluate metrics. Stop when performance plateaus or target metrics are met.
5. **Evaluation Metrics:** Track accuracy/precision/recall per class, AUC, and if applicable fairness metrics (e.g. demographic parity). Also measure synthetic data quality (e.g. statistical closeness to real data distributions) as in SDV.
6. **Deployment:** Package as a Python toolkit (possibly a Jupyter-based interface) that data scientists can run. Include logging of each iteration's model metrics. Optionally provide a

simple web UI or command-line for specifying parameters (which classes to augment, etc.).

Expected Challenges & Mitigation: Synthetic data may introduce artifacts or biases. We will mitigate this by conservative generation (e.g. limit number of synthetic samples, use validation sets to detect overfitting). Ensuring privacy (if real data is sensitive) may require differential privacy techniques. Model retraining could overfit to synthetic quirks; we'll interleave real and synthetic data. Computation may be heavy if iterating many times; we will use a small batch feedback loop and stop early when gains become marginal.

Required Resources & Skills:

- **Data:** A medium-sized tabular dataset with known imbalance (e.g. public healthcare datasets, Kaggle benchmarks).
- **Models:** Familiarity with generative models (GANs, VAEs) and ML frameworks (PyTorch/TensorFlow or SDV library).
- **Skills:** Python programming, ML, statistics. Knowledge of imbalanced learning and model evaluation techniques is critical.

Success Criteria: The project will be successful if the tool demonstrably **improves model performance** on held-out data by $\geq 10\%$ compared to baseline on an imbalanced task, or significantly enhances metrics on underrepresented groups. Deliverables include code for the pipeline, a demo on a real dataset, and documentation.

Novelty Justification: While synthetic data tools exist (e.g. SDV) and closed-loop augmentation has been studied, an **open-source, integrated toolkit** that loops from error analysis to targeted generation is not yet available. Our project uniquely combines error-driven feedback with synthetic sampling to adaptively improve models. This bridges a gap between static augmentation and continual ML training, inspired by recent notions of “closed-loop” synthetic data generation but focusing on tabular domains.

Sub-Tasks (prioritized):

1. **Build Baseline Pipeline:** Data ingestion, model training, and error analysis module.
2. **Implement Synthetic Generators:** Integrate one or more data generators (GAN or SDV) and test static augmentation.
3. **Develop Feedback Loop:** Code the loop that identifies errors and adds synthetic samples, then retrains and evaluates.

Recommended Sources: Review the Synthetic Data Vault library and documentation. Key papers on generative models for tabular data (TVAE, Gaussian copula, CopulaGAN). SDV (MIT) tutorials and InterpretML documentation (for error analysis). Benchmark datasets (UCI or Kaggle) for imbalanced tasks.